Phase 6: Planning, Memory & Context

This document provides the complete implementation for **Phase 6: Planning, Memory & Context** for the **Banking — AI Banking Support & Advisory Agent (Non-Transactional)** project. In earlier phases, the banking AI system evolved through: • Rule-based workflows • LLM reasoning • Retrieval-Augmented Generation (RAG) • Safe tool usage Phase 6 improves the agent further by introducing: • Multi-step reasoning • Planning logic • Short-term memory • Long-term memory • Context-aware conversations • Multi-turn interaction handling

Banking Safety Constraints

- The agent must never execute banking transactions.
- The agent must never store raw sensitive customer data.
- Memory must avoid retaining PII.
- High-risk conversations must escalate to humans.
- Conversation memory must expire after defined limits.
- Planning logic must respect banking compliance rules.

Why Planning & Memory are Needed

Earlier versions of the banking AI assistant treated each user query independently. This caused several problems: • The agent forgot previous user questions. • Follow-up conversations felt unnatural. • Multi-step banking guidance was difficult. • Users had to repeat information repeatedly. • Complex banking workflows could not be tracked. Phase 6 solves these issues using memory and planning systems.

1. Planning & Memory Architecture

User Query ↓ Conversation State Manager ↓ Memory Retrieval ↓ Planning Engine ↓ Tool + RAG Usage ↓ LLM Reasoning ↓ Safety Validation ↓ Final Banking Response

2. Multi-Step Planning Logic

The banking AI assistant now performs multi-step reasoning before responding. Instead of directly answering, the system: 1. Understands user intent 2. Determines whether tools are needed 3. Retrieves banking policies 4. Checks safety constraints 5. Generates a grounded response

```
def create_plan(user_query):

    plan = []

    if "loan" in user_query.lower():
        plan.append("Retrieve loan policy")

    if "fraud" in user_query.lower():
        plan.append("Retrieve fraud procedure")

    plan.append("Validate banking safety")

    plan.append("Generate response")

    return plan
```

3. Example Planning Flow

User Query: "I lost my debit card and also want to know if fraudulent transactions are reversible." The planning engine decomposes the task into multiple sub-steps:

- Retrieve debit card blocking procedure
- Retrieve fraud reporting guidelines
- Retrieve disputed transaction policy
- Combine results into one response
- Recommend escalation if necessary

4. Short-Term Conversation Memory

Short-term memory stores recent conversation history during the active session. This allows the banking agent to understand follow-up questions.

```
conversation_memory = []

def save_message(role, content):

    conversation_memory.append({
        "role": role,
        "content": content
    })

save_message("user", "How do I report fraud?")

save_message(
    "assistant",
    "You can contact the fraud helpline."
)

print(conversation_memory)
```

5. Multi-Turn Conversation Example

Without memory: User: "How do I report fraud?" Assistant: "Call the fraud helpline." User: "Can I also freeze my card?" Old agent problem: The system forgets the earlier fraud discussion. With memory: The assistant understands the user is still discussing the fraud scenario.

6. Long-Term Memory

Long-term memory stores persistent non-sensitive user preferences. Examples: • Preferred language • Preferred banking channel • Frequently asked banking topics The system must NEVER store: • Account numbers • PINs • Aadhaar/PAN numbers • Passwords

```
long_term_memory = {
    "preferred_language": "English",
    "preferred_channel": "Mobile App",
    "interest_topic": "Credit Cards"
}
```

7. Memory Retention & Reset Rules

Memory handling rules are critical for banking compliance. Retention policies: • Session memory expires after inactivity. • Sensitive information is never persisted. • Users may request memory reset. • Risky conversations are

cleared immediately.

```
MAX_MEMORY_ITEMS = 10

def trim_memory(memory):

    if len(memory) > MAX_MEMORY_ITEMS:
        memory.pop(0)

    return memory
```

8. Memory Reset Handling

Users or compliance systems may request conversation memory deletion.

```
def reset_memory():

    global conversation_memory

    conversation_memory = []

    return "Conversation memory cleared."
```

9. Context-Aware Response Generation

The assistant now combines: • Current query • Previous conversation history • Retrieved banking documents • Safety rules to generate more natural and contextual responses.

```
def build_context(user_query):

    recent_messages = conversation_memory[-3:]

    context = "\n".join([
        msg["content"]
        for msg in recent_messages
    ])

    return f'''
    Conversation Context:
    {context}

    Current User Query:
    {user_query}
    '''
```

10. Improved Conversation Quality

The following comparison demonstrates the improvement from memory-enabled conversations.

Scenario	Without Memory	With Memory
Fraud discussion	Forgets earlier context	Maintains fraud discussion continuity
Loan eligibility follow-up	Asks repeated questions	Uses previous answers
Branch recommendation	No personalization	Uses stored preferred city
Multi-step requests	Incomplete handling	Tracks all subtasks

11. Multi-Turn Testing

The system was tested using realistic banking conversations.

```
USER: "How do I report fraud?"

ASSISTANT:
"Please contact the fraud helpline immediately."

USER:
"Can I also block my debit card?"

ASSISTANT:
"Yes. Since you mentioned fraud earlier,
you should immediately freeze or block
your debit card using the banking app
or customer support helpline."
```

12. New Failure Modes Introduced

Adding memory and planning introduces new risks: • Incorrect memory retention • Stale contextual information • Memory leakage across users • Over-reliance on old conversation history • Increased token usage

Failure Mode	Example	Mitigation
Memory Leakage	Another user's data appears	Session isolation
Stale Context	Old loan discussion reused	Memory expiration
Excessive Context	Very long prompts	Memory summarization
Sensitive Data Retention	Stored account number	PII filtering

13. Final Memory-Enabled Banking Agent

The final Phase 6 banking AI assistant combines: • Multi-step planning • RAG retrieval • Tool usage • Short-term memory • Long-term preferences • Context-aware conversations • Banking safety compliance

```
def banking_memory_agent(user_query):

    save_message("user", user_query)

    plan = create_plan(user_query)

    context = build_context(user_query)

    response = f'''
    PLAN:
    {plan}

    CONTEXT:
    {context}
    '''

    save_message("assistant", response)

    return response
```

Phase 6 transformed the Banking AI Support Agent into a context-aware conversational system. The agent can now: • Remember previous conversation context • Handle multi-turn banking conversations • Perform multi-step planning • Retain safe long-term preferences • Improve personalization • Maintain banking safety and compliance This phase significantly improved the realism and usability of the banking assistant.